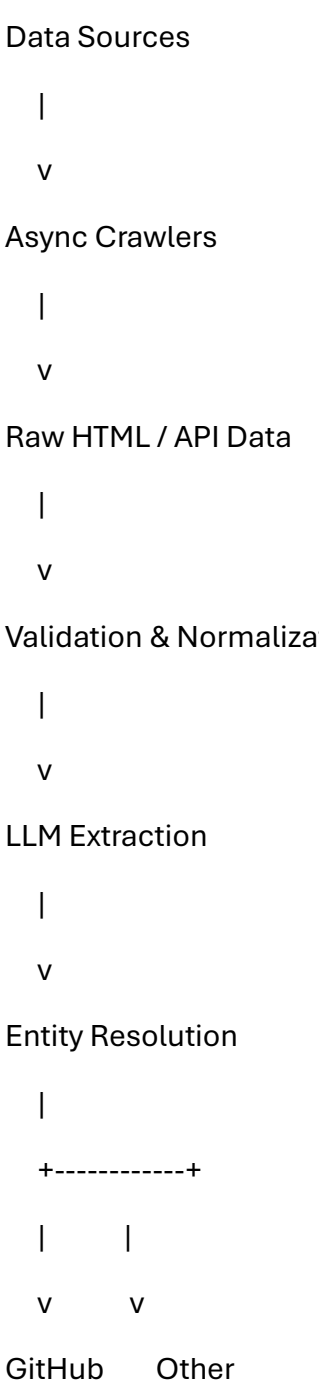
AI Intelligence Pipeline – System Architecture

1. Overview

The AI Intelligence Pipeline is an asynchronous data ingestion and enrichment system designed to collect intelligence from multiple sources, including startups, products, research papers, jobs, and AI news.

The pipeline separates crawling, extraction, validation, enrichment, entity resolution, and storage.

2. System Architecture



Enrichment Enrichment

| |

+----------+

|

v

JSON / CSV Storage

3. Data Collection

The system uses asynchronous HTTP requests to collect data efficiently.

The main sources include:

- ArXiv research papers
- GitHub repositories
- Startup sources
- Job listings
- AI news sources

aiohttp is used for asynchronous HTTP communication and connection pooling.

4. Scalability – 500,000+ Records

For large-scale production use, the crawler can be distributed across multiple workers.

Message Queue

Redis / Kafka

|

+----------+----------+

| | |

Worker 1 Worker 2 Worker N

| | |

+----------+----------+

|

Database

The workers are stateless, allowing additional workers to be added horizontally.

Concurrency limits and connection pools prevent excessive requests to individual websites.

5. HTTP 429 Rate Limit Handling

When a website returns HTTP 429 Too Many Requests, the system should:

1. Read the Retry-After header when available.
2. Wait before retrying.
3. Use exponential backoff.
4. Add random jitter.
5. Limit the number of retry attempts.
6. Send persistent failures to a dead-letter queue.

Example:

delay = min(base × 2^attempt + jitter, maximum_delay)

This prevents repeated requests from overwhelming the source.

6. HTTP 413 Payload Handling

Large documents should not be directly sent to an LLM.

The pipeline handles large documents using:

Large Document

|

v

HTML Cleaning

|

v

Text Extraction

|

v

Semantic Chunking

|

v

Individual LLM Requests

|

v

Result Merging

This reduces payload size and prevents context-window limitations.

7. LLM Fallback Architecture

The extraction layer supports multiple LLM providers.

LLM Request

|

v

Gemini

|

Failure

|

v

Groq

|

Failure

|

v

DeepSeek

Each provider is accessed through a common interface.

If one provider fails, the pipeline attempts the next provider rather than terminating the entire process.

8. Freshness Tracking

Each record receives a timestamp such as:

collectedAt

publishedDate

Records can be filtered using a freshness window.

For example:

published_at >= current_time - 24 hours

A deterministic fingerprint can also be generated using:

source + canonical URL

or:

source + title + publication date

This helps prevent duplicate processing.

9. Storage Strategy

PostgreSQL can be used as the primary production database because it supports:

- Transactions
- Indexing
- Unique constraints
- JSONB
- Deduplication

Additional infrastructure can include:

PostgreSQL → Structured records

S3 → Raw HTML/documents

Redis → Cache/rate limiting

Kafka → Distributed queues

Neo4j → Entity relationships

Vector DB → Semantic search

10. Entity Resolution

Entity resolution normalizes different representations of the same organization.

For example:

Open AI

OpenAI Inc.

OpenAI, Inc.

can resolve to:

OpenAI

The system uses deterministic matching and canonical mappings.

Unknown entities are preserved rather than being invented.

11. GitHub Enrichment

Research papers are enriched by searching GitHub for relevant repositories.

For matched repositories, the pipeline collects:

GitHub URL

Repository owner

Repository name

GitHub stars

Current trial result:

Research papers: 1,000

GitHub repositories found: 154

12. Data Quality

The pipeline follows these principles:

- Preserve the original source URL.
- Validate extracted records.
- Normalize dates and entity names.
- Avoid duplicate records.
- Do not fabricate missing information.
- Use LLMs for extraction and normalization rather than generating unsupported records.

13. Current Trial Results

Dataset	Records
Research Papers	1,000
GitHub Repositories	154

Dataset	Records
Jobs	175
News	19
Startups	81
Entity Mapping	81

Product extraction was implemented, but additional source validation and pagination are required before reaching the requested 1,000 validated product records.

14. Production Improvements

For production deployment, the system can be improved with:

- Distributed crawler workers
- Redis/Kafka queues
- PostgreSQL
- S3 object storage
- Neo4j entity graph
- Vector database
- Prometheus/Grafana monitoring
- Dead-letter queues
- Automated retry policies
- Containerized workers
- Scheduled crawling
- Centralized logging

15. Conclusion

The AI Intelligence Pipeline provides a modular architecture for collecting, validating, enriching, resolving, and exporting AI intelligence data.

The asynchronous architecture allows the system to scale to significantly larger datasets while maintaining rate-limit protection, fault tolerance, data freshness, and data quality.